

Workshop 6 | 5CS046

1. Create a program which applies a sepia filter on an image. This is the process of recalculating each pixel's RGB values using the sepia conversion formula.

Formula:

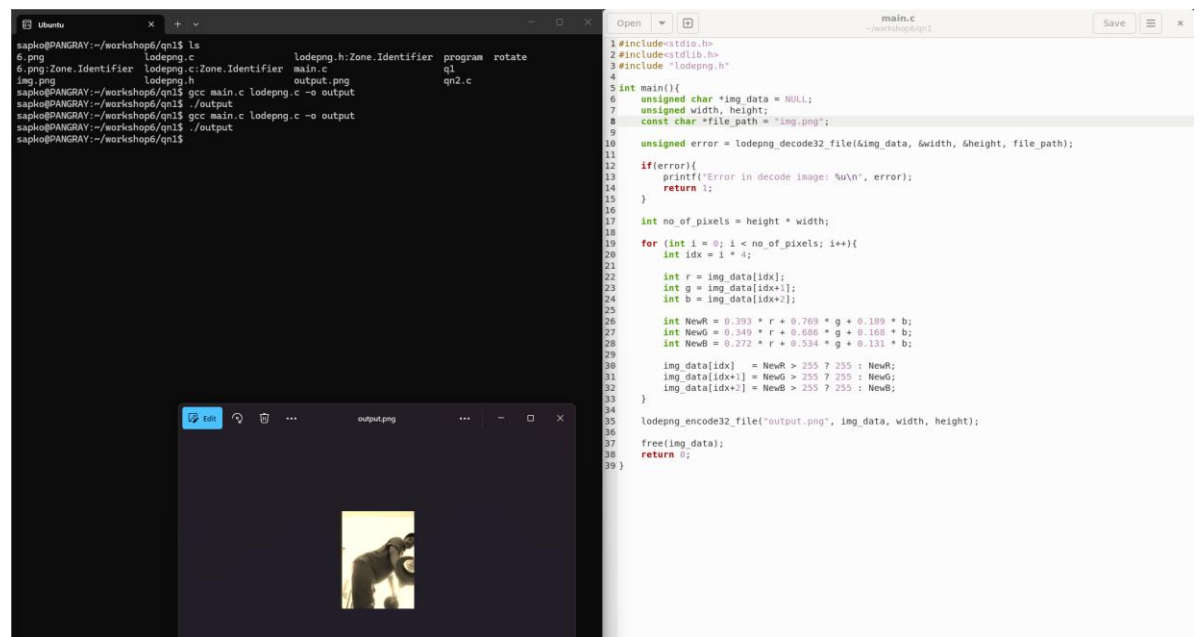
$$\text{NewR} = 0.393\text{R} + 0.769\text{G} + 0.189\text{B}$$

$$\text{NewG} = 0.349\text{R} + 0.686\text{G} + 0.168\text{B}$$

$$\text{NewB} = 0.272\text{R} + 0.534\text{G} + 0.131\text{B}$$

If any value exceeds 255, set it to 255.

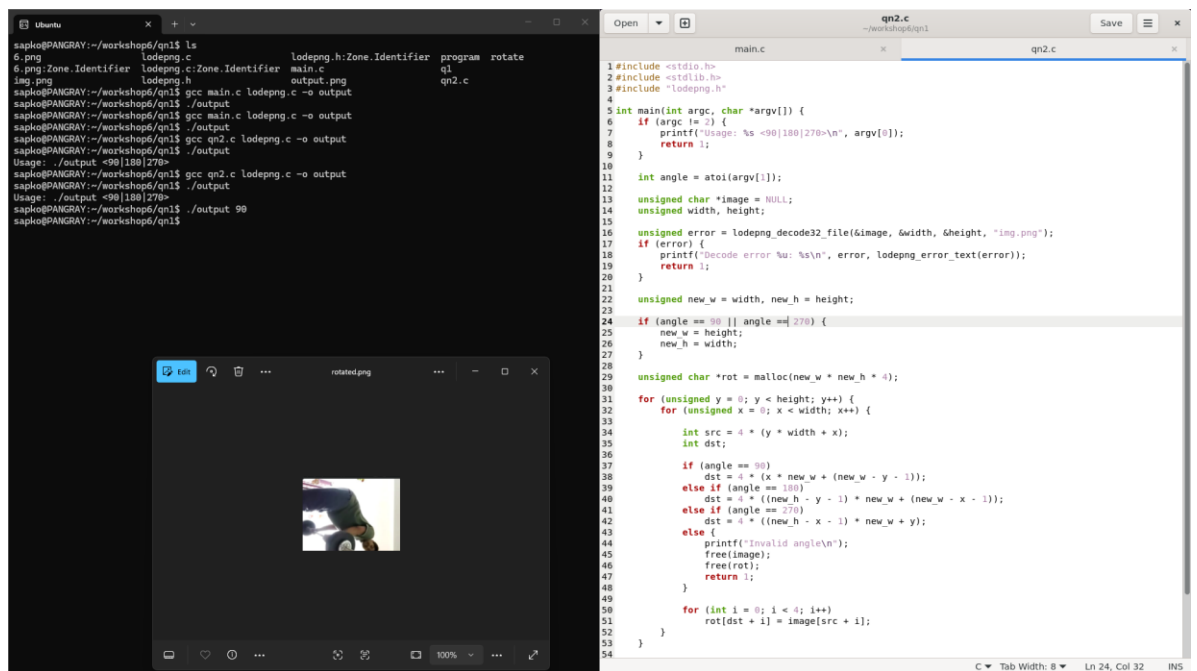
NOTE: 0 is minimum and 255 is maximum.



The screenshot shows a terminal window on the left and a code editor on the right. The terminal displays the execution of the program, showing the file 'img.png' being processed and the output 'output.png' being generated. The code editor shows the implementation of the sepia filter in C, using the libpng library. The code includes headers for stdio, stdlib, and libpng, and defines a main function that reads an image, applies the sepia filter, and saves the result.

```
1#include<stdio.h>
2#include<stdlib.h>
3#include "lodepng.h"
4
5int main(){
6    unsigned char *img_data = NULL;
7    unsigned width, height;
8    const char *file_path = "img.png";
9
10    unsigned error = lodepng_decode32_file(&img_data, &width, &height, file_path);
11
12    if(error){
13        printf("Error in decode image: %u\n", error);
14        return 1;
15    }
16
17    int no_of_pixels = height * width;
18
19    for (int i = 0; i < no_of_pixels; i++){
20        int idx = i * 4;
21
22        int r = img_data[idx];
23        int g = img_data[idx+1];
24        int b = img_data[idx+2];
25
26        int NewR = 0.393 * r + 0.769 * g + 0.189 * b;
27        int NewG = 0.349 * r + 0.686 * g + 0.168 * b;
28        int NewB = 0.272 * r + 0.534 * g + 0.131 * b;
29
30        img_data[idx] = NewR > 255 ? 255 : NewR;
31        img_data[idx+1] = NewG > 255 ? 255 : NewG;
32        img_data[idx+2] = NewB > 255 ? 255 : NewB;
33    }
34
35    lodepng_encode32_file("output.png", img_data, width, height);
36
37    free(img_data);
38    return 0;
39}
```

2. Create a program which rotates an image file (90/180/270 degrees) depending on the command line argument using "argv."



3. Create a program which adjusts the brightness of an image file depending on the command line argument using “argv.” The user should enter a positive value to increase brightness or a negative value to decrease brightness.

Formula:

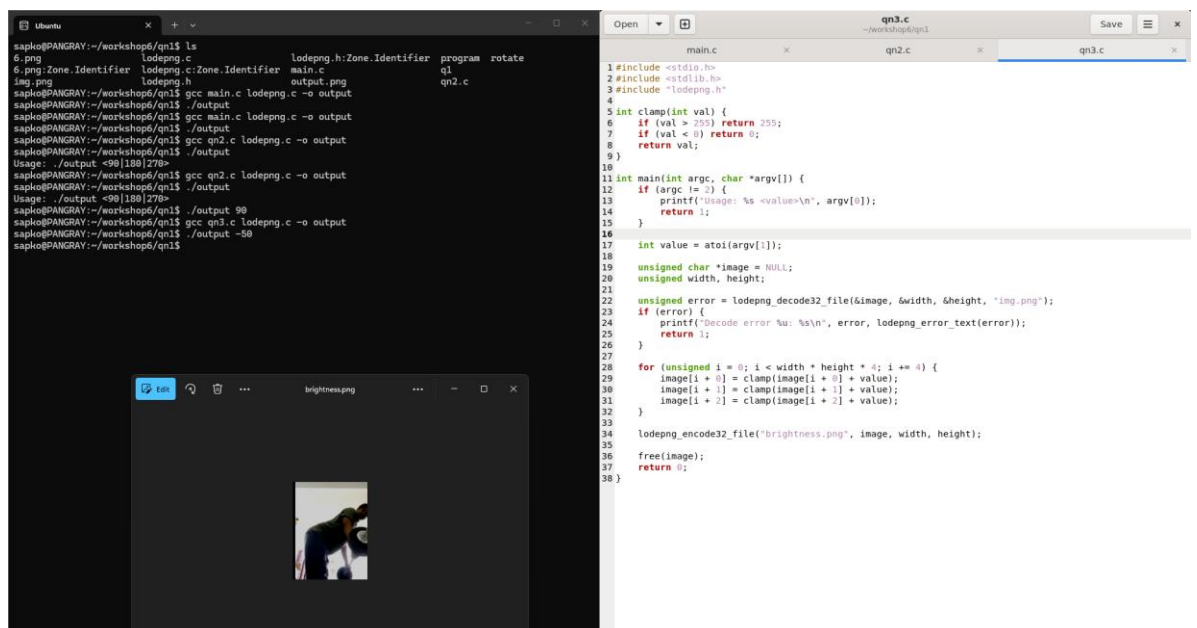
$R = R + \text{value}$

$G = G + \text{value}$

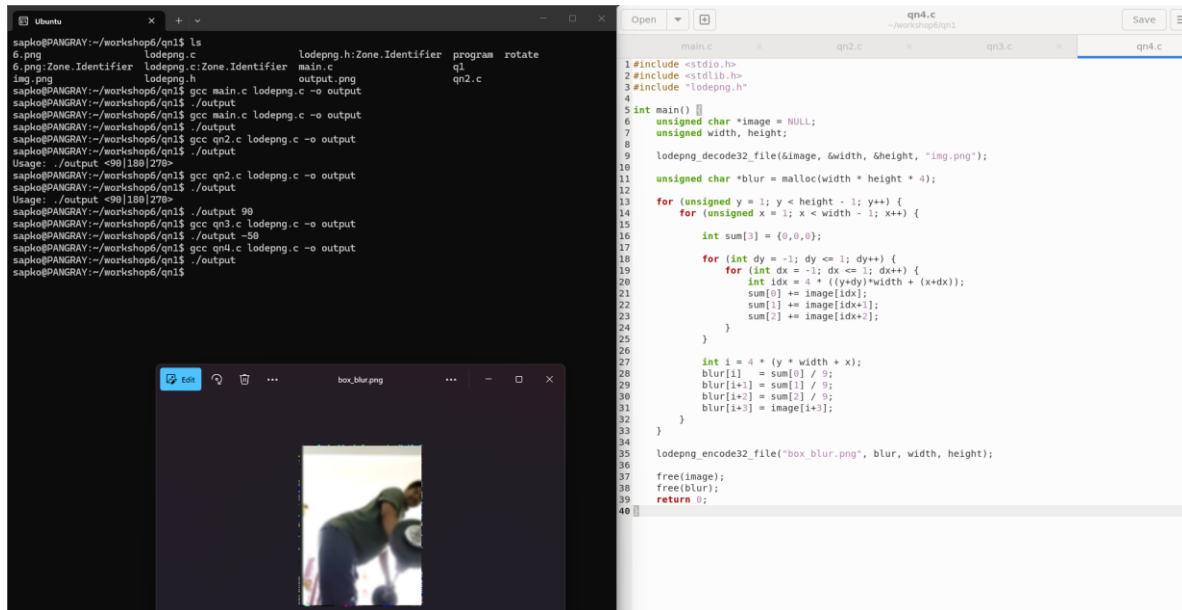
$B = B + \text{value}$

If any value becomes greater than 255, set it to 255.

If any value becomes less than 0, set it to 0.



4. Research different types of blurring techniques for images and apply at least 2 using the lodepng library.
 - a. Box blur
 - b. Gaussian blur



The screenshot displays a development environment with a terminal window on the left and a code editor on the right. The terminal shows the execution of a C program that applies a box blur to an image. The code editor shows the implementation of the box blur function in C, which uses the lodepng library for image handling.

```
sapko@PANGRAY:~/workshop/qn1$ ls
6.png  Zone.Identifier  lodepng.h  Zone.Identifier  program  rotate
img.png  lodepng.h
sapko@PANGRAY:~/workshop/qn1$ gcc main.c lodepng.c -o output
sapko@PANGRAY:~/workshop/qn1$ ./output
Usage: ./output <90|180|270>
sapko@PANGRAY:~/workshop/qn1$ gcc qn2.c lodepng.c -o output
sapko@PANGRAY:~/workshop/qn1$ ./output
Usage: ./output <90|180|270>
sapko@PANGRAY:~/workshop/qn1$ gcc qn3.c lodepng.c -o output
sapko@PANGRAY:~/workshop/qn1$ ./output
Usage: ./output <90|180|270>
sapko@PANGRAY:~/workshop/qn1$ gcc qn4.c lodepng.c -o output
sapko@PANGRAY:~/workshop/qn1$ ./output
sapko@PANGRAY:~/workshop/qn1$
```

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include "lodepng.h"
4
5 int main() {
6     unsigned char *image = NULL;
7     unsigned width, height;
8
9     lodepng_decode32_file(&image, &width, &height, "img.png");
10
11     unsigned char *blur = malloc(width * height * 4);
12
13     for (unsigned y = 1; y < height - 1; y++) {
14         for (unsigned x = 1; x < width - 1; x++) {
15
16             int sum[3] = {0, 0, 0};
17
18             for (int dy = -1; dy <= 1; dy++) {
19                 for (int dx = -1; dx <= 1; dx++) {
20                     int idx = 4 * ((y+dy)*width + (x+dx));
21                     sum[0] += image[idx];
22                     sum[1] += image[idx+1];
23                     sum[2] += image[idx+2];
24                 }
25             }
26
27             int i = 4 * (y * width + x);
28             blur[i] = sum[0] / 9;
29             blur[i+1] = sum[1] / 9;
30             blur[i+2] = sum[2] / 9;
31             blur[i+3] = image[i+3];
32         }
33     }
34
35     lodepng_encode32_file("box_blur.png", blur, width, height);
36
37     free(image);
38     free(blur);
39     return 0;
40 }
```